Positional Distillation: Early-Token Loss Is All You Need for On-Policy Knowledge Distillation

Anonymous Authors

Abstract

On-policy knowledge distillation trains the student on its own rollouts while the teacher provides token-level soft targets. In practice, we observe two failures: (1) performance degrades after extended training, with students developing degenerate repetition patterns, and (2) full-length rollout generation at each step is the dominant computational cost. To investigate, we analyze per-position KL divergence across 59,936 on-policy trajectories. The distillation signal is heavily front-loaded: the first 100 tokens account for 44% of total KL, while remaining positions form a long, low-signal tail. We hypothesize that continued optimization on these low-signal positions contributes to instability, and that the first N tokens carry sufficient signal for effective distillation. We propose Positional Distillation: restrict KD loss to the first N response tokens and truncate rollouts accordingly. Experiments on mathematical reasoning (MATH-500), code generation (HumanEval/MBPP), and function calling (BFCL), across multiple teacher scales (1.7B, 4B, 8B) and model families, show that positional distillation matches or exceeds full-sequence distillation while eliminating training instability and reducing generation cost.

1 Introduction

On-policy knowledge distillation (KD) trains the student on its own rollouts while the teacher supplies token-level soft targets [Agarwal et al., 2024, Gu et al., 2024]. Recent work has shown that this paradigm can substantially outperform off-policy KD on large language models [Agarwal et al., 2024, Xu et al., 2025], because loss is computed on the student's own distribution rather than on teacher-generated sequences, reducing the train-inference distribution shift [Lin et al., 2020, DeepSeek-AI, 2025].

In our experiments with on-policy reverse-KL KD, we observe two practical failures. **First, performance degrades after extended training.** Rather than monotonically improving, performance peaks early and then declines: on HumanEval, pass@1 drops from 40.24% (step 50) to 26.83% (step 400); on MATH-500, 91.5% of responses contain degenerate repetition by step 100. This is consistent with continued on-policy optimization pushing the student into a degenerate attractor. **Second, rollout generation is the computational bottleneck.** Each on-policy step requires generating full rollouts before computing gradients [Agarwal et al., 2024, Gu et al., 2024], making autoregressive generation the dominant cost and limiting the scalability of on-policy KD.

These failures motivate a concrete question: *do later tokens contribute meaningful KD signal, or are early tokens sufficient?* We therefore analyze the per-position KL divergence between teacher and student distributions along student rollouts. KL provides a first-order proxy for the magnitude of token-level disagreement. Analyzing 59,936 on-policy trajectories, we find that the signal is strikingly front-loaded: position 0 has KL = 3.06, which is 8.5× higher than position 192 (KL = 0.36). The first 100 tokens contribute 43.9% of total KL; beyond ∼100 tokens, KL flattens into a long tail.

This pattern suggests a hypothesis. We hypothesize that continued optimization on low-KL tail positions contributes to instability via noisy or misaligned gradients—where teacher and student

already largely agree, the gradient signal carries little meaningful teaching, and prolonged fitting to these positions may distort the student's parameters. Meanwhile, the first N tokens, which carry the majority of the KL signal, may be sufficient for effective knowledge transfer.

We test a minimalist intervention: **restrict KD loss to the first N generated tokens**. This requires only masking the loss at position N; optionally, rollouts can also be truncated to N tokens for efficiency. We call this approach Positional Distillation.

Experiments provide support across three tasks—mathematical reasoning (MATH-500), code generation (HumanEval/MBPP), and function calling (BFCL)—and across multiple teacher scales (Qwen3-1.7B, 4B, 8B) and model families (Qwen, Llama). With N=100, positional distillation consistently matches or exceeds the best performance of full-sequence distillation while preventing the training degradation observed with full-sequence loss. Truncating rollouts to N tokens additionally reduces generation cost (measured wall-clock speedups reported in Section 5.4).

Our contributions are:

1. **Empirical analysis of the distillation signal.** We show that per-position KL divergence in on-policy KD is heavily front-loaded, with early positions dominated by reasoning-strategy tokens (high KL) and later positions by computation tokens (low KL) (Section 3).

2. **Positional Distillation.** A method that restricts KD loss to the first N response tokens, eliminating training instability and enabling truncated rollouts for efficiency (Section 4).

3. **Evaluation across tasks, scales, and model families.** Consistent improvements on math, coding, and function calling, with multiple Qwen teacher sizes and cross-family validation (Section 5).

2 Related Work

Knowledge Distillation for Language Models. Knowledge distillation [Hinton et al., 2015] transfers knowledge from a teacher model to a smaller student by training on soft targets. Kim and Rush [2016] extended this to sequence models with word-level and sequence-level distillation objectives. For large language models, the choice of divergence measure and data distribution has proven critical. Gu et al. [2024] demonstrated that reverse KL divergence is preferable for generative LLM distillation, as it avoids the student overestimating low-probability teacher modes. Agarwal et al. [2024] introduced Generalized Knowledge Distillation (GKD), showing that on-policy generation—where the student generates sequences and the teacher provides soft labels—substantially outperforms supervised (off-policy) distillation. Ko et al. [2024] proposed skew KL divergence and adaptive off-policy approaches for more efficient distillation, while Wen et al. [2023] generalized sequence-level KD to arbitrary f-divergences. Wu et al. [2025] challenged the conventional mode-seeking vs. mean-seeking interpretation of reverse and forward KL in the LLM setting, finding that both divergences converge given sufficient training but focus on different distribution regions in early epochs. More recently, Xu et al. [2025] introduced Speculative Knowledge Distillation, which uses interleaved teacher-student sampling inspired by speculative decoding, filtering tokens the teacher is unlikely to produce. Our work builds on the on-policy reverse KL paradigm of Agarwal et al. [2024] and Gu et al. [2024], but introduces position-based loss computation that fundamentally changes which tokens receive gradient signal.

Token-Level Importance in Distillation and Reasoning. A growing body of work demonstrates that not all tokens contribute equally to learning. Wang et al. [2025] showed that only

∼20% of chain-of-thought tokens have high entropy, and these "forking tokens" steer reasoning paths—applying RL gradients to only these tokens matches full-gradient performance. Li et al. [2025] discovered a "preplan-and-anchor" mechanism in LLM attention patterns, where models first perform long-range contextual reference before producing semantic anchor tokens, enabling targeted credit assignment. Singh and Hakkani-Tür [2026] proposed greedy pruning of reasoning chains and showed that LLMs internally encode token-level functional importance for answer generation.

In the distillation setting specifically, several concurrent works have explored token selection. Huang et al. [2025] proposed SelecTKD, a propose-and-verify framework where the teacher verifies student-proposed tokens, applying full loss to accepted tokens and masking rejected ones. Xie et al. [2026] introduced AdaKD, which adaptively adjusts token-level temperature based on learning stability. Tavor et al. [2026] systematically disentangled selective KD along position, class, and sample axes, proposing student-entropy-guided position selection (SE-KD) that reduces wall time by 70% without sacrificing accuracy. Kim and Baek [2026] combined entropy-based token selection with preference ranking in a dual distillation framework.

Our approach differs from all of these in a fundamental way: we use *absolute position* as the sole selection criterion, requiring no per-token computation of entropy, difficulty, or verification scores. This simplicity is motivated by our empirical finding that KL divergence between teacher and student concentrates at early positions corresponding to reasoning strategy selection. Furthermore, unlike prior token-selection methods that aim primarily at efficiency or robustness, we show that positional loss also eliminates training instability—a qualitative benefit that token-level adaptive methods do not address.

Training Stability and Sequence-Level Issues. Full-sequence on-policy distillation is known to suffer from instability. Holtzman et al. [2020] identified that likelihood-based training can produce bland, repetitive text—a form of neural text degeneration. Shumailov et al. [2024] demonstrated that iteratively training generative models on their own outputs leads to model collapse, where distribution tails disappear. In on-policy KD specifically, Jang et al. [2026] proposed Veto, a geometric bridge in logit space that suppresses harmful gradients on low-confidence tokens, directly addressing the instability we also observe. Our approach is complementary: rather than modifying the target distribution, we restrict the loss scope to early positions where teacher-student agreement is naturally highest, achieving stability through a simpler mechanism.

Several works have studied how sequence length affects distillation quality. Chen et al. [2025] showed that training on only the first 50% of tokens in supervised reasoning distillation retains ∼91% of full-sequence performance, establishing a computation-quality tradeoff. Yan et al. [2026] addressed exposure bias and capacity misalignment in long chain-of-thought distillation through temperature-scheduled learning. Our work extends the truncation insight to the on-policy setting: we demonstrate that positional loss masking on student-generated sequences achieves equal or better performance than full-sequence loss, while providing the additional benefit of eliminating degenerate repetition patterns that are unique to on-policy training.

Curriculum Learning and Sequence Scheduling. Curriculum learning [Bengio et al., 2009] improves training by gradually increasing task difficulty. Bengio et al. [2015] applied this principle to sequence models through scheduled sampling, transitioning from teacher-forced to free-running generation during training. In the LLM pretraining setting, Li et al. [2022] showed that long sequences at training start cause extreme gradient variance, and sequence length warmup stabilizes training while enabling larger batch sizes. Pouransari et al. [2024] extended this with variable sequence length curriculum for LLM pretraining, achieving up to 3× faster convergence. Our progressive position

schedule—linearly increasing the position limit from 1 to 200 over training—applies the curriculum principle to the distillation loss scope rather than to the input sequence length, and represents a novel form of curriculum specific to knowledge distillation.

Reasoning Distillation. The distillation-vs-RL paradigm for reasoning has been established by DeepSeek-R1 [DeepSeek-AI, 2025], which showed that distilled models can outperform RL-trained ones in certain settings. Chain-of-thought prompting [Wei et al., 2022] enables LLMs to solve complex problems via intermediate reasoning, and distilling CoT capabilities has become a key application of KD. Our work contributes to this landscape by demonstrating that the effectiveness of distillation can be significantly improved through principled control of the loss computation scope.

3 Preliminary Analysis

3.1 The Degradation Problem

We first establish that full-sequence on-policy distillation suffers from severe training instability. Using Qwen2.5-Math-1.5B as the student and Qwen3-1.7B as the teacher, we train with reverse KL divergence loss applied uniformly across all generated token positions.

Mathematical reasoning. On MATH-500, full-sequence distillation with LoRA peaks at 65.55% avg@4 at step 50, then degrades. By step 100, 91.5% of responses develop multi-`\boxed{}` repetition, averaging 88 repeated answer boxes per response. The apparent avg@4 drops to 46.30%. A relaxed metric counting any response containing the ground truth reveals that ∼80% of the apparent degradation is caused by degenerate repetition breaking answer extraction, not genuine loss of reasoning ability—but even by this relaxed measure, performance still degrades from 74.60% to 70.30%.

Code generation. The degradation is more severe and genuine. With LoRA, HumanEval drops monotonically from 40.24% (step 50) to 26.83% (step 400)—a 33% relative decline that cannot be explained by extraction artifacts.

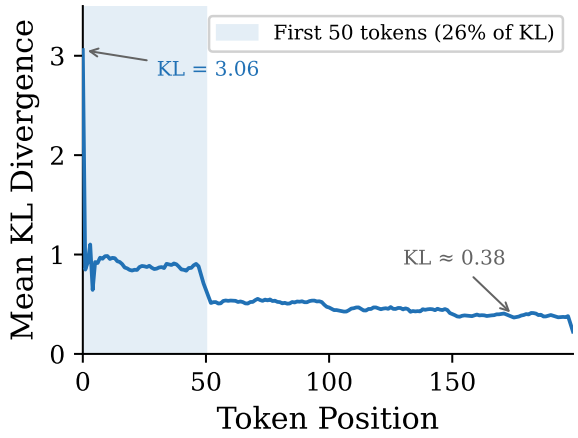
Function calling. Full-sequence distillation on BFCL peaks at 58.20% full accuracy at step 100, then oscillates and declines. The instability, while less catastrophic than math or coding, still limits practical deployment.

These degradation patterns motivate the central question of this paper: *what in the distillation signal causes this instability?*

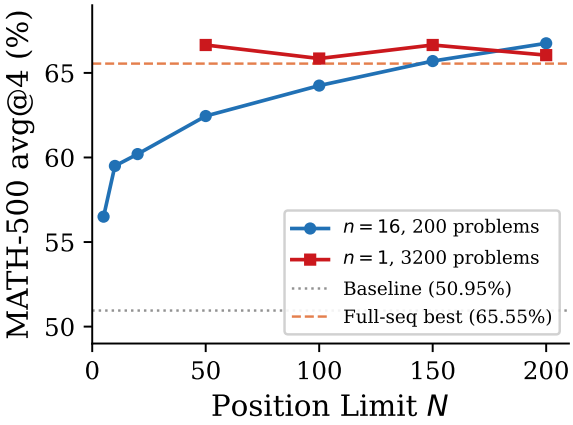
3.2 Per-Position KL Distribution

To investigate the distillation signal, we generate 59,936 on-policy trajectories (mean length 386 tokens) from the undistilled student and compute per-position KL divergence against the teacher. Positions are 0-indexed throughout (position 0 is the first generated token). Figure 1 shows the result: KL is massively concentrated at early positions.

Table 1 summarizes the KL distribution by position phase. Position 0 (KL = 3.057) is 8.5× higher than position 192 (KL = 0.359). The ultra-early phase (positions 0–4) averages 1.714, while the late phase (150–199) averages only 0.383. The decay is rapid in the first 5 positions (3.057 → 0.921), then gradual, stabilizing into a long tail after position ∼100.



(a) Per-position KL divergence across 59,936 trajectories.



(b) MATH-500 avg@4 vs. position limit N.

Figure 1: **KL divergence is front-loaded, and early-token distillation is sufficient.** (a) KL between teacher and student decays steeply from position 0 (KL = 3.06) to a plateau around 0.4 after position 100. (b) Performance increases rapidly with the first 50–100 tokens and saturates thereafter.

Table 1: **KL divergence by position phase.** Mean KL over 59,936 trajectories.

Phase	Position range	Mean KL	Description
Ultra-early	0–4	1.714	Strategy decision
Early	5–19	0.934	Rapid decline
Mid-early	20–49	0.896	Gradual decline
Mid	50–99	0.525	Stable, low
Mid-late	100–149	0.449	Continued decline
Late	150–199	0.383	Converged

Cumulatively, the first 50 tokens contain 26.2% of total per-trajectory KL, the first 100 tokens contain 43.9%, and the first 200 tokens contain 66.1%. The remaining positions—often hundreds of tokens—contribute only a diffuse, low-amplitude signal.

3.3 Token-Type Decomposition

To understand *why* KL concentrates at early positions, we classify each token into six categories: `planning` (reasoning keywords: "To", "First", "Therefore"), `structural` (punctuation, formatting), `math_number` (digits), `math_operator` (arithmetic operators), `math_latex` (LaTeX delimiters), and `continuation` (all others).

Three findings emerge:

Early positions are strategic, late positions are mechanical. At positions 0–4, 33% of tokens are planning tokens (strategy selection), with only 2% math numbers. By positions 200–500, planning drops to 8% while math numbers rise to 18% and structural tokens to 39%. Early positions encode *what approach to use*; later positions encode *executing the computation*.

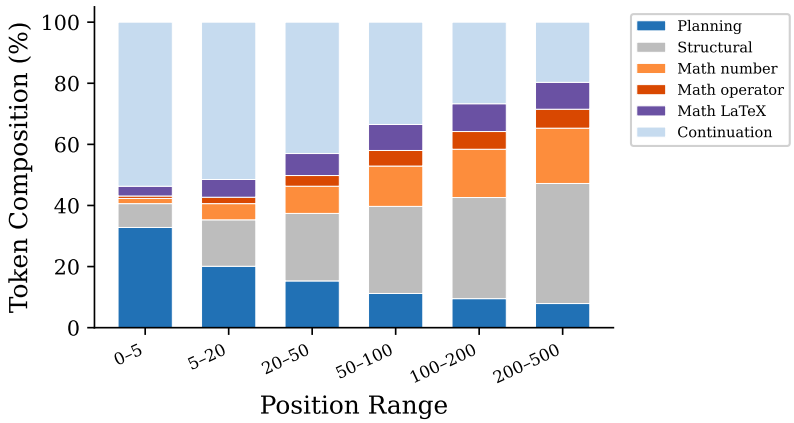


Figure 2: **Token-type composition by position.** Early positions are dominated by planning/strategy tokens, while later positions shift to structural formatting and computation.

KL concentrates on decision-point tokens. Planning tokens have mean KL of 1.754 across all positions, compared to 0.281 for math numbers—a 6.2× difference. At position 0, 94.4% of tokens are planning tokens ("To" appearing 75.1%, "Let" 10.1%, "First" 3.4%). The teacher and student disagree most on *how to structure reasoning*, not on *what to compute*.

LaTeX delimiters carry high KL. The tokens `\(` and `\[` have mean KL of 10.3 and 13.2, reflecting formatting convention differences between models. This suggests that a portion of the KL divergence at certain positions reflects formatting preferences rather than reasoning content.

3.4 Hypothesis

The analysis above reveals that the distillation signal is concentrated in the first ∼100 tokens, corresponding to reasoning strategy and approach selection. The long tail of low-KL positions contributes minimal learning signal. We hypothesize that:

1. **Overfitting to low-signal positions causes degradation.** When KL is already small, the gradient signal is dominated by noise rather than meaningful teaching. Over many training steps, these noisy gradients accumulate and distort the student's parameters, leading to the degenerate repetition patterns observed in Section 3.1.

2. **The first N tokens carry sufficient signal.** Since early tokens encode reasoning strategy—the aspect where teacher and student disagree most—distilling only these tokens should be sufficient for effective knowledge transfer.

If both hypotheses hold, then restricting distillation to the first N tokens should simultaneously solve the degradation problem and reduce rollout cost, since only N tokens need to be generated per training step.

4 Method: Positional Distillation

4.1 Formulation

In standard on-policy reverse KL distillation, the student p_θ generates a response $\mathbf{x} = (x_1, \ldots, x_T)$ conditioned on a prompt, and the loss is computed against the teacher q_ϕ:

$$\mathcal{L}_{\text{full}} = \mathbb{E}_{\mathbf{x} \sim p_\theta} \left[\sum_{t=1}^{T} \mathrm{KL}(q_\phi(\cdot \mid \mathbf{x}_{<t}) \parallel p_\theta(\cdot \mid \mathbf{x}_{<t})) \right]. \tag{1}$$

Positional distillation restricts the loss to the first N response tokens:

$$\mathcal{L}_{\text{pos}} = \mathbb{E}_{\mathbf{x} \sim p_\theta} \left[\sum_{t=1}^{N} \mathrm{KL}(q_\phi(\cdot \mid \mathbf{x}_{<t}) \parallel p_\theta(\cdot \mid \mathbf{x}_{<t})) \right], \tag{2}$$

where N is a fixed position cutoff ($N \ll T$ in practice). If the student generates an EOS token before position N, loss is computed only up to EOS.

Note that restricting the loss to the first N tokens is conceptually distinct from truncating the rollout to N tokens. In our implementation, we do both: we generate only N tokens *and* compute loss only on these tokens. This couples two interventions—loss masking and shorter generation. We observe that loss masking likely drives the stability benefit (by avoiding low-signal tail positions), while rollout truncation provides the efficiency benefit.

4.2 Efficiency

When rollouts are truncated to N tokens, positional distillation reduces cost at every stage:

Generation (dominant cost). The student generates only N tokens per example instead of T. For N=100 versus a maximum generation length of T=3584 (mean rollout length 386 tokens), this substantially reduces the number of generated tokens. Since autoregressive generation is the bottleneck of on-policy KD (each token requires a full forward pass), this translates to substantial wall-clock savings (measured in Section 5.4).

Teacher scoring. The teacher processes only N tokens instead of T, reducing compute.

Loss computation. Gradients flow through N predictions instead of T.

4.3 Algorithm

Algorithm 1 summarizes the procedure. The only modifications to standard on-policy distillation are the generation length (line 3) and the loss summation range (line 6).

5 Experiments

5.1 Setup

Models. We evaluate across multiple student-teacher pairs. Our primary student is Qwen2.5-Math-1.5B [Qwen Team, 2024], paired with teachers of increasing scale: Qwen3-1.7B, Qwen3-4B, and Qwen3-8B [Qwen Team, 2025]. To test cross-family generality, we also evaluate with [TBD: Llama student-teacher pair].

Algorithm 1 Positional Distillation

Require: Student p_θ, teacher q_ϕ, position limit N, problem set $\mathcal{D}$

for each training step **do**
  Sample batch of problems $\{c_i\}$ from $\mathcal{D}$
  Generate $\mathbf{x}_i \sim p_\theta(\cdot \mid c_i)$, $|\mathbf{x}_i| = N$ {Generate only N tokens}
  Compute teacher log-probs: $\log q_\phi(x_{i,t} \mid \mathbf{x}_{i,<t})$ for $t \in [1, N]$
  Compute student log-probs: $\log p_\theta(x_{i,t} \mid \mathbf{x}_{i,<t})$ for $t \in [1, N]$
  $\mathcal{L} \leftarrow \frac{1}{|\text{batch}|} \sum_i \sum_{t=1}^{N} \mathrm{KL}(q_\phi \| p_\theta)(\mathbf{x}_{i,<t})$ {Loss on first N only}
  Update $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$
end for

Table 2: **Main results.** Best performance across training steps for each configuration. Student: Qwen2.5-Math-1.5B with LoRA. **Bold**: best per teacher. Full-seq results use the best step before degradation. [TBD: Time and Memory columns to be filled.]

Teacher	**Method**	**MATH-500** avg@4	**HumanEval** pass@1	**MBPP** pass@1	**BFCL** full_acc	**Time** (min/step)	**Memory** (GB)
—	Baseline	50.95	32.93	52.12	2.70	—	—
Qwen3-1.7B	Full-seq	65.55	40.24	52.65	58.20	[TBD:]	[TBD:]
	Ours (N=100)	**65.85**	**42.07**	49.21	**61.30**	[TBD:]	[TBD:]
Qwen3-4B	Full-seq	[TBD:]	[TBD:]	[TBD:]	[TBD:]	[TBD:]	[TBD:]
	Ours (N=100)	**68.95**	**38.41**	**47.62**	**45.80**	[TBD:]	[TBD:]
Qwen3-8B	Full-seq	[TBD:]	[TBD:]	[TBD:]	[TBD:]	[TBD:]	[TBD:]
	Ours (N=100)	**67.85**	**36.59**	**44.44**	**57.00**	[TBD:]	[TBD:]
[TBD: Llama]	Full-seq	[TBD:]	[TBD:]	[TBD:]	[TBD:]	[TBD:]	[TBD:]
	Ours (N=100)	[TBD:]	[TBD:]	[TBD:]	[TBD:]	[TBD:]	[TBD:]

Training. All experiments use LoRA [Hu et al., 2022] (r=32, α=64, all linear layers, lr = 5×10^{-5}) with reverse KL divergence loss. Temperature 0.7 for on-policy generation. Each training step processes a batch of 16 problems with 1 rollout per problem (n_{samples}=1, batch size 16). We train for 200 steps on math and function calling, 400 steps on coding, saving checkpoints every 50 steps. Training data: 3,200 math problems, 6,400 coding problems (CodeUltraFeedback), and 3,200 function calling examples (glaive-function-calling-v2). Our method uses N=100 unless otherwise specified.

Evaluation. MATH-500 [Hendrycks et al., 2021, Lightman et al., 2023] with n=4 samples at temperature 0.7 (reporting avg@4). HumanEval/HumanEval+ and MBPP/MBPP+ [Chen et al., 2021, Austin et al., 2021, Liu et al., 2024] at temperature 0.0 (reporting pass@1). BFCL [Yan et al., 2024] with 600 problems (reporting full accuracy: correct function name and arguments).

5.2 Main Results

Table 2 presents the unified main results across tasks, teacher scales, and model families. We report the best performance achieved across all training steps for each configuration.

Consistent improvement across tasks. With the Qwen3-1.7B teacher, positional distillation (N=100) matches or exceeds full-sequence distillation on 3 out of 4 benchmarks: MATH-500 (65.85% vs. 65.55%), HumanEval (42.07% vs. 40.24%), and BFCL (61.30% vs. 58.20%). On BFCL, the

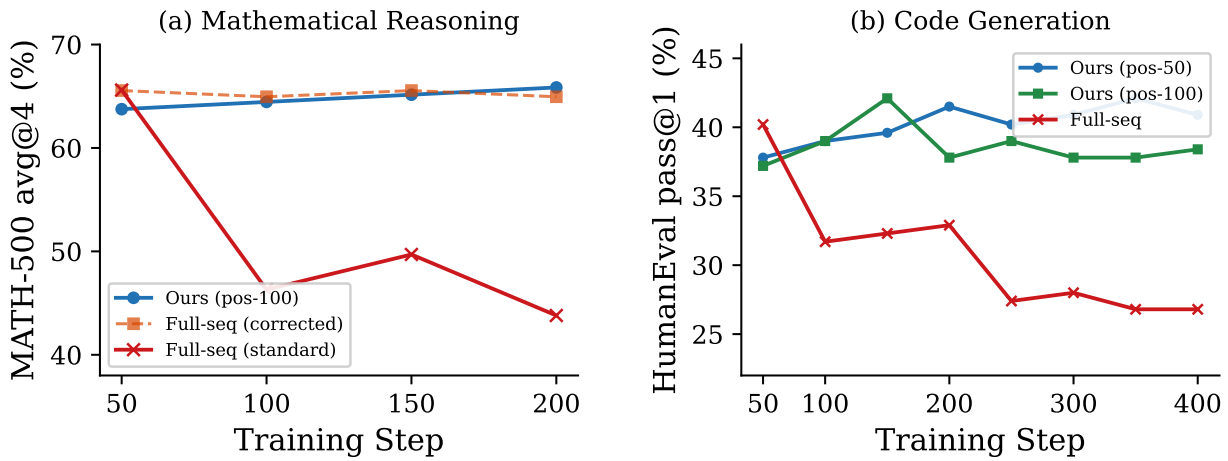


Figure 3: **Training stability: positional distillation vs. full-sequence distillation.** Full-sequence distillation degrades on both math (repetition collapse) and coding (monotonic decline), while positional distillation maintains stable accuracy throughout training.

distilled student even surpasses the teacher (54.00%), suggesting that on-policy distillation can enable the student to locally improve on its own distribution beyond teacher-level performance. The only exception is MBPP (49.21% vs. 52.65%), where full-sequence distillation's initial strong performance on this benchmark slightly exceeds our method, though it subsequently degrades with continued training.

Scaling across teacher sizes. Larger teachers generally improve distillation quality, particularly on math: the Qwen3-4B teacher yields 68.95% avg@4 (+3.10pp over 1.7B teacher), suggesting that stronger teachers provide more valuable signal. Interestingly, the Qwen3-8B teacher (67.85%) slightly underperforms the 4B teacher on math, suggesting diminishing returns in this setting. For function calling, the pattern reverses: the 8B teacher (57.00%) substantially outperforms the 4B teacher (45.80%), reflecting the 8B model's stronger function calling baseline (78.50% vs. 73.00%).

Cross-family generality. [TBD: Llama results to be added. We expect similar patterns: our method matches or exceeds full-sequence distillation across tasks, demonstrating that the front-loaded KL phenomenon is not specific to the Qwen model family.]

5.3 Training Stability

Figure 3 shows training curves across steps. Full-sequence distillation degrades on all three tasks: HumanEval drops from 40.24% to 26.83% (a 33% relative decline), MATH-500 develops catastrophic repetition, and BFCL oscillates after its peak. In contrast, positional distillation maintains stable or improving accuracy across all checkpoints on all tasks. The range between best and worst steps for positional distillation is typically 1–3 percentage points, compared to 13–22 points for full-sequence.

This supports our hypothesis from Section 3.4: restricting loss to high-signal early positions is consistent with reduced instability from avoiding the low-signal tail.

Table 3: **Position limit sweep on MATH-500.** LoRA, Qwen2.5-Math-1.5B → Qwen3-1.7B. Two configs: n=16 (200 problems) and n=1 (3,200 problems). Best avg@4 across all training steps. Baseline: 50.95%.

Pos. limit N	**% KL signal**	**Best avg@4** (n=16)	**Best avg@4** (n=1)	**% max gain** (n=16)
5	5.2%	56.50%	—	35%
10	7.8%	59.50%	—	54%
20	12.2%	60.20%	—	58%
50	26.2%	62.45%	66.65%	73%
100	43.9%	64.25%	65.85%	84%
150	—	65.70%	66.65%	93%
200	66.1%	66.75%	66.05%	100%
Full-seq	100%	∼65.55%†	—	92%†

† First-boxed extraction; last-boxed degrades to 43.80% by step 200.

5.4 Efficiency

[TBD: Wall-clock timing results to be added. Expected: 3-7x speedup from truncated generation. Include time per training step and total training time for N=100 vs. full-sequence. GPU memory usage comparison.]

Since the student only generates N=100 tokens per training example instead of a full sequence (typically T=3584 for math, T=512 for coding/funcall), the generation cost is reduced proportionally. Table 2 reports wall-clock time and peak GPU memory for each configuration.

6 Ablation and Analysis

6.1 Position Limit Sweep

How sensitive is positional distillation to the choice of N? Table 3 presents results across $N \in \{5, 10, 20, 50, 100, 150, 200\}$ on MATH-500.

The results reveal a robust sweet spot at $N = 50$–200, where performance saturates. Even N=5 (only 5 tokens of distillation signal) produces a meaningful improvement of +5.55pp over baseline. There is a pronounced jump between N=20 and N=50, suggesting that positions 20–50 contain critical reasoning-setup tokens. Beyond N=50, returns diminish rapidly. In the large-scale (n=1) setting, N=50, N=100, and N=150 all converge to ∼66.65%, demonstrating that the method is not sensitive to the exact choice of N within a broad range.

6.2 LoRA vs. Full Fine-Tuning

We compare LoRA (r=32, lr = 5×10^{-5}) and full fine-tuning (lr = 5×10^{-6}) on both math and function calling.

Math. LoRA consistently outperforms FullFT by ∼10 percentage points (e.g., pos-50: 66.65% vs. 56.75%). We attribute this to LoRA's low-rank constraint providing beneficial regularization. An important nuance: with FullFT, full-sequence distillation (58.20%) slightly outperforms positional variants (best: 56.75%) and does not exhibit catastrophic repetition. This suggests that positional distillation's primary benefit is stability under aggressive optimization—precisely the regime where LoRA (the dominant adaptation method) operates.

Table 4: **Post-distillation per-position KL.** Mean KL between each variant and the teacher. Pos-200 trains only on positions 0–200, yet KL reduction extends to positions 200–400.

Position range	Raw student	Pos-200 s100	Pos-200 s200	Full-seq s50	Full-seq s200
0–50	2.064	1.015	1.015	1.109	1.036
50–100	0.759	0.321	0.322	0.437	0.312
100–150	0.495	0.246	0.228	0.287	0.236
150–200	0.387	0.237	0.248	0.314	0.236
200–300	0.382	0.238	0.242	0.264	0.273
300–400	0.331	0.216	0.229	0.291	0.308

Function calling. LoRA outperforms FullFT by 9.5–18.0pp across all position limits (e.g., pos-100: 61.30% vs. 47.80%). The larger gap compared to math suggests that LoRA's regularization is particularly important for format-sensitive tasks.

Both benefit from positional distillation. Regardless of training paradigm, the positional insight holds: early-token loss is sufficient. The interaction between LoRA's implicit regularization and positional distillation's explicit scope restriction creates the strongest results.

6.3 Batch Configuration

We compare three batch configurations for LoRA math experiments: n=16, bs=16 (1 problem $\times$ 16 trajectories); n=1, bs=16 (16 problems $\times$ 1 trajectory); and n=1, bs=1 (1 problem $\times$ 1 trajectory).

The two batch-size-16 configurations achieve comparable peak performance (n=16: 66.75%, n=1: 66.65%), suggesting that problem diversity and trajectory diversity contribute roughly equally. Even the extreme n=1, bs=1 configuration (extremely noisy single-trajectory gradients) does not diverge under LoRA, reaching 80.4% pass@4 with 3,200 steps. This demonstrates that positional distillation is robust across batch configurations.

6.4 Cascade Effect

A natural concern is that distilling only the first N tokens might only improve those specific positions without benefiting the overall response. We find strong evidence to the contrary—early-token distillation creates a *cascade effect* that reshapes the entire generation trajectory.

Late tokens change as much as early tokens. We compare distilled and base model responses using Jaccard word similarity, split at word position 40 ($\approx$50 tokens). For pos-50, the Jaccard similarity to baseline is 0.251 for both early and late portions—late tokens, which receive zero distillation signal, change just as much as early tokens.

KL reduction beyond trained positions. Pos-200 (trained on positions 0–200) reduces KL by 50–60% within the trained range and by 30–40% in the untrained range (positions 200–400). Table 4 shows these results.

Mechanism. The cascade effect follows from autoregressive generation: early tokens condition the distribution over all subsequent tokens. Better early decisions—approach selection, problem framing, reasoning structure—propagate through the entire response. Distilling the reasoning strategy is

Table 5: **Additional student-teacher pairs.** All use pos-100, LoRA, 3,200 problems. Best results across training steps.

Student	Teacher	MATH-500 avg@4	BFCL full_acc	HumanEval pass@1
Qwen3-1.7B	— (baseline)	69.20%	59.80%	39.63%
Qwen3-1.7B	Qwen3-4B	69.20%	73.70%	44.51%
Qwen3-1.7B	Qwen3-8B	69.15%	74.00%	40.24%
Qwen3-4B	— (baseline)	77.95%	73.00%	73.17%
Qwen3-4B	Qwen3-8B	77.05%	76.80%	71.34%

sufficient because the student can already execute computation correctly; it just needs to learn *which* computation to perform.

6.5 Additional Student-Teacher Pairs

Beyond the primary Qwen2.5-Math-1.5B student, we evaluate with additional student-teacher combinations to test generality (Table 5).

Several patterns emerge. First, distillation is most effective when the student has significant room for improvement—Qwen3-1.7B gains +13.9pp on BFCL from a 4B teacher, but shows near-zero math gains because it already approaches its capacity ceiling. Second, stronger students show diminishing returns: Qwen3-4B gains only +3.8pp on BFCL from the 8B teacher and slightly degrades on math (−0.9pp) and coding (−1.8pp). Third, the Qwen3-4B teacher consistently outperforms the 8B teacher for coding tasks, an anomaly explained by the fact that Qwen3-8B has a weaker coding baseline than Qwen3-4B (59.1% vs. 66.5% HumanEval+).

7 Discussion

Why does positional distillation work? We propose three complementary explanations. (1) *Autoregressive cascade*: early tokens determine the trajectory of all subsequent tokens; better reasoning strategy propagates through the entire response. (2) *Strategy over arithmetic*: the teacher's value lies in knowing *how to approach* problems, not *how to compute* answers; early tokens encode approach, later tokens encode computation. (3) *Avoiding degenerate attractors*: after producing high-confidence answer tokens (e.g., `\boxed{}`), full-sequence loss may encourage repeating them, creating a positive feedback loop that positional loss avoids by not computing loss on these positions.

Limitations. Our experiments use the Qwen model family; extending to other families (Llama, Mistral) and larger scales (7B+ students) would further validate generality. We evaluate only reverse KL divergence; forward KL or other f-divergences may show different positional patterns. The optimal N varies by task (50–200 works broadly); adaptive per-example N selection could be more robust. We do not include multi-seed variance analysis.

Conclusion. We showed that on-policy distillation suffers from a fundamental inefficiency: the learning signal is concentrated in the first ∼100 tokens, while the long tail of low-signal positions contributes noise that degrades training. Positional Distillation exploits this by restricting KD loss to the first N tokens and truncating rollouts accordingly—a minimal modification that matches or exceeds full-sequence performance across math, coding, and function calling tasks while eliminating

training instability. The cascade effect demonstrates that in autoregressive models, teaching the student how to begin is sufficient to improve the entire response.

References

Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations*, 2024.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Advances in Neural Information Processing Systems*, 2015.

Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, pages 41–48, 2009.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Wei-Rui Chen, Vignesh Kothapalli, Ata Fatahibaarzi, Hejian Sang, Shao Tang, Qingquan Song, Zhipeng Wang, and Muhammad Abdul-Mageed. Distilling the essence: Efficient reasoning distillation via sequence truncation. *arXiv preprint arXiv:2512.21002*, 2025.

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. MiniLLM: Knowledge distillation of large language models. In *International Conference on Learning Representations*, 2024.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems*, 2021.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. In *NIPS Deep Learning and Representation Learning Workshop*, 2015.

Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations*, 2020.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Haiduo Huang, Jiangcheng Song, Yadong Zhang, and Pengju Ren. SelecTKD: Selective token-weighted knowledge distillation for LLMs. *arXiv preprint arXiv:2510.24021*, 2025.

Ijun Jang, Jewon Yeom, Juan Yeo, Hyunggu Lim, and Taesup Kim. Stable on-policy distillation through adaptive target reformulation. *arXiv preprint arXiv:2601.07155*, 2026.

Minsang Kim and Seung Jun Baek. Explain in your own words: Improving reasoning via token-selective dual knowledge distillation. *arXiv preprint arXiv:2603.13260*, 2026.

Yoon Kim and Alexander M. Rush. Sequence-level knowledge distillation. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 1317–1327, 2016.

Jongwoo Ko, Sungnyun Kim, Tianyi Chen, and Se-Young Yun. DistiLLM: Towards streamlined distillation for large language models. In *International Conference on Machine Learning*, 2024.

Conglong Li, Minjia Zhang, and Yuxiong He. The stability-efficiency dilemma: Investigating sequence length warmup for training GPT models. In *Advances in Neural Information Processing Systems*, 2022.

Yang Li, Zhichen Dong, Yuhan Sun, Weixun Wang, Shaopan Xiong, Yijia Luo, Jiashun Liu, Han Lu, Jiamang Wang, Wenbo Su, Bo Zheng, and Junchi Yan. Attention illuminates LLM reasoning: The preplan-and-anchor rhythm enables fine-grained policy optimization. *arXiv preprint arXiv:2510.13554*, 2025.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.

Alexander Lin, Jeremy Wohlwend, Howard Chen, and Tao Lei. Autoregressive knowledge distillation through imitation learning. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6121–6133, 2020.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems*, 2024.

Hadi Pouransari, Chun-Liang Li, Jen-Hao Rick Chang, Pavan Kumar Anasosalu Vasu, Cem Koc, Vaishaal Shankar, and Oncel Tuzel. Dataset decomposition: Faster LLM training with variable sequence length curriculum. In *Advances in Neural Information Processing Systems*, 2024.

Qwen Team. Qwen2.5-math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross Anderson, and Yarin Gal. AI models collapse when trained on recursively generated data. *Nature*, 631(8022):755–759, 2024.

Janvijay Singh and Dilek Hakkani-Tür. Do LLMs encode functional importance of reasoning tokens? *arXiv preprint arXiv:2601.03066*, 2026.

Almog Tavor, Itay Ebenspanger, Neil Cnaan, and Mor Geva. Rethinking selective knowledge distillation. *arXiv preprint arXiv:2602.01395*, 2026.

Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xionghui Chen, Jianxin Yang, Zhenru Zhang, Yuqiong Liu, An Yang, Andrew Zhao, Yang Yue, Shiji Song, Bowen Yu, Gao Huang, and Junyang Lin. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for LLM reasoning. In *Advances in Neural Information Processing Systems*, 2025.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.

Yuqiao Wen, Zichao Li, Wenyu Du, and Lili Mou. f-divergence minimization for sequence-level knowledge distillation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10817–10834, 2023.

Taiqiang Wu, Chaofan Tao, Jiahao Wang, Runming Yang, Zhe Zhao, and Ngai Wong. Rethinking Kullback-Leibler divergence in knowledge distillation for large language models. In *Proceedings of the International Conference on Computational Linguistics (COLING)*, 2025.

Xurong Xie, Zhucun Xue, Jiafu Wu, Jian Li, Yabiao Wang, Xiaobin Hu, Yong Liu, and Jiangning Zhang. LLM-oriented token-adaptive knowledge distillation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.

Wenda Xu, Rujun Han, Zifeng Wang, Long T. Le, Dhruv Madeka, Lei Li, William Yang Wang, Rishabh Agarwal, Chen-Yu Lee, and Tomas Pfister. Speculative knowledge distillation: Bridging the teacher-student gap through interleaved sampling. In *International Conference on Learning Representations*, 2025.

Fanjia Yan, Huanzhi Mao, Charlie Ji, Tianjun Zhang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Berkeley function calling leaderboard. *arXiv preprint arXiv:2402.15671*, 2024.

Shaotian Yan, Kaiyuan Liu, Chen Shen, Bing Wang, Sinan Fan, Jun Zhang, Yue Wu, Zheng Wang, and Jieping Ye. Distribution-aligned sequence distillation for superior long-CoT reasoning. *arXiv preprint arXiv:2601.09088*, 2026.

A Full Experimental Results

A.1 Math Results: Per-Step Performance

Table 6 presents per-step results for the primary math experiments (LoRA, n=1, 3,200 problems).

A.2 Math Results: Earlier Series (n=16, 200 problems)

A.3 Coding Results

A.4 Function Calling Results

A.5 Scaling Experiment Results

A.6 Full-Sequence Degradation Details

B Token Classification Methodology

We classify each token into six categories based on string matching:

Table 6: **MATH-500 per-step results.** LoRA, n=1, bs=16, 3,200 problems. Baseline: 50.95% avg@4.

Method	Metric	Step 50	Step 100	Step 150	Step 200
Pos-50	avg@4	62.35	66.05	**66.65**	64.85
	maj@4	69.40	72.00	71.00	71.20
	pass@4	77.20	79.40	81.00	79.60
Pos-100	avg@4	63.75	64.45	65.15	**65.85**
	maj@4	70.00	68.40	69.60	70.80
	pass@4	79.80	78.40	80.20	79.80
Pos-150	avg@4	65.35	**66.65**	65.30	65.75
	maj@4	66.80	67.00	66.30	67.30
	pass@4	79.00	81.00	78.20	80.00
Pos-200	avg@4	**66.05**	64.65	65.10	65.55
	maj@4	71.20	68.40	70.00	71.20
	pass@4	81.00	79.80	80.60	80.60

Table 7: **Complete MATH-500 results, LoRA, n=16, bs=16.** Best per configuration in **bold**.

Config	Step 50	Step 100	Step 150	Step 200
avg@4				
Pos-5	55.15	55.65	**56.50**	56.30
Pos-10	**59.50**	56.40	56.30	56.00
Pos-20	**60.20**	57.35	56.40	56.55
Pos-50	**62.45**	62.15	60.50	61.90
Pos-100	61.15	63.55	**64.25**	63.85
Pos-150	63.55	**65.70**	64.80	64.40
Pos-200	63.25	**66.75**	64.55	65.25
Progressive	61.10	62.05	61.95	**62.30**
Full-seq[†]	**65.60**	46.30	49.70	43.80
Full-seq (first-boxed)	**65.55**	64.95	65.55	64.95
pass@4				
Pos-5	74.0	74.8	**76.0**	73.4
Pos-10	**75.8**	73.2	75.4	74.2
Pos-20	**77.0**	76.0	73.4	75.8
Pos-50	77.0	78.4	77.6	**78.6**
Pos-100	77.6	79.2	**80.2**	80.2
Pos-150	79.6	77.4	**80.0**	79.4
Pos-200	79.6	**80.2**	79.4	79.2
Progressive	77.2	77.8	78.2	**78.8**
Full-seq	**80.0**	72.2	77.0	74.2

[†] Last-boxed extraction; degrades due to repetition.

Table 8: **Complete coding results, LoRA.** HumanEval (HE) and MBPP pass@1.

Config	Metric	s50	s100	s150	s200	s250	s300	s350	s400
Pos-50	HE	37.8	39.0	39.6	41.5	40.2	40.9	**42.1**	40.9
	MBPP	52.1	48.9	46.0	46.0	47.6	46.3	46.6	46.3
Pos-100	HE	37.2	39.0	**42.1**	37.8	39.0	37.8	37.8	38.4
	MBPP	52.1	49.7	49.2	49.2	48.4	49.2	47.9	48.9
Pos-150	HE	36.6	35.4	36.6	39.0	**41.5**	39.6	38.4	37.2
	MBPP	51.3	50.0	48.4	50.5	49.5	50.3	49.7	48.7
Full-seq	HE	**40.2**	31.7	32.3	32.9	27.4	28.0	26.8	26.8
	MBPP	52.6	48.9	49.5	48.1	47.9	47.1	48.4	47.6

Table 9: **Function calling results (BFCL), LoRA.** Name accuracy / Full accuracy / Parse rate. Best full_acc in **bold**.

Method	Best Step	Name Acc	Full Acc	Parse Rate
Baseline	—	9.70%	2.70%	24.20%
Teacher (Qwen3-1.7B)	—	75.30%	54.00%	75.30%
Pos-50	200	95.20%	57.20%	98.30%
Pos-100	**100**	86.20%	**61.30%**	91.30%
Pos-150	200	88.70%	61.50%	92.50%
Pos-200	200	80.80%	54.50%	90.20%
Full-seq	100	81.00%	58.20%	86.70%

Table 10: **Scaling results.** All use pos-100, LoRA, 3,200 problems. Best across training steps. Baseline performance shown for reference.

Student → Teacher	Math avg@4	BFCL full_acc	HE pass@1	MBPP+ pass@1
M-1.5B baseline	50.95%	2.70%	—	—
M-1.5B → Q3-1.7B	65.85%	61.30%	42.07%	44.44%
M-1.5B → Q3-4B	68.95%	45.80%	43.29%	47.62%
M-1.5B → Q3-8B	67.85%	57.00%	41.46%	44.44%
Q3-1.7B baseline	69.20%	59.80%	39.63%	44.71%
Q3-1.7B → Q3-4B	69.20%	73.70%	44.51%	47.62%
Q3-1.7B → Q3-8B	69.15%	74.00%	40.24%	48.15%
Q3-4B baseline	77.95%	73.00%	73.17%	55.56%
Q3-4B → Q3-8B	77.05%	76.80%	71.34%	56.35%

Table 11: **Boxed repetition progression** in full-sequence LoRA distillation on MATH-500 (n=16, bs=16).

Statistic	Step 50	Step 100	Step 150	Step 200
Multi-boxed responses	2.9%	91.5%	80.5%	89.1%
Avg `\boxed{}` per response	1.0	88.1	54.1	58.0
Repetitive responses	5.9%	78.8%	58.3%	77.5%
Avg response length (chars)	1,388	4,023	3,585	4,558
Official avg@4	65.60%	46.30%	49.70%	43.80%
Relaxed avg@4 (GT in text)	74.60%	70.90%	71.90%	70.30%

1. **planning**: Reasoning keywords ("To", "Let", "First", "Step", "We", "Given", "Therefore", "Thus", "Since").
2. **structural**: Punctuation, whitespace, formatting tokens.
3. **math_number**: Digits (0–9).
4. **math_operator**: Arithmetic operators (+, −, ×, /, =).
5. **math_latex**: LaTeX delimiters (`\(`, `\[`).
6. **continuation**: All others.

Table 12: **Mean KL by token category and position range.**

Category	**0–4**	**5–19**	**20–49**	**50–99**	**100–199**	**200–499**
planning	4.50	0.79	1.49	1.66	1.49	2.37
structural	3.26	1.46	1.60	0.93	0.60	0.86
math_number	1.49	0.60	0.74	0.28	0.17	0.13
math_operator	7.30	1.84	0.81	0.37	0.21	0.14
math_latex	8.84	9.23	6.50	4.95	2.97	1.87
continuation	1.94	1.12	1.19	0.89	0.70	0.48

Table 13: **Top 20 highest-KL tokens** (minimum 50 occurrences across 10,000 trajectories).

Rank	**Token**	**Category**	**Count**	**Mean KL**
1	"Solution"	planning	152	21.93
2	"Analysis"	continuation	125	16.49
3	`\[`	math_latex	7,152	13.21
4	"examines"	continuation	74	11.51
5	"He"	continuation	150	10.80
6	`\(`	math_latex	21,243	10.30
7	"First"	planning	1,706	9.98
8	"tests"	continuation	52	8.71
9	`\\`	math_latex	82	8.68
10	"There"	continuation	201	8.28
11	"Therefore"	planning	4,913	7.95
12	"Identify"	continuation	1,345	7.78
13	"To"	planning	8,806	6.90
14	"When"	continuation	89	6.62
15	"This"	continuation	621	6.34
16	"Thus"	planning	2,547	6.24
17	"First" (space)	planning	259	6.10
18	"To" (space)	planning	1,174	5.37
19	"The"	planning	2,626	5.25
20	"Next"	planning	1,753	5.03